

# Code Review and Engineering Collaboration

## The Purpose Of Code Review

The primary purpose of code review is to improve the health of the codebase over time. A reviewer is not looking for perfection; they are deciding whether the change definitely improves the overall state of the system, even if it is not flawless.

A secondary purpose is shared understanding. Every review spreads context about a part of the system to at least one more person, which reduces the risk carried by any single engineer leaving.

## What Reviewers Should Look For

Design comes first. A change that is well written but architecturally wrong is more expensive to unwind than one that is structurally sound but needs tidying, so structural concerns should be raised before style.

Reviewers should confirm that the change does what the author claims, that it is covered by tests that would fail if the behaviour regressed, and that naming will still make sense to someone reading it in a year.

Comments should explain why something matters rather than only what to change. A comment that gives the reasoning teaches; one that gives only an instruction has to be repeated on the next change.

## Handling Disagreement In Review

Technical facts and data overrule opinion and personal preference. Where neither side has data, the established convention of the codebase generally wins over individual style preference.

If a disagreement cannot be resolved in review comments, escalate to a conversation rather than continuing to exchange written arguments. Long comment threads tend to harden positions rather than resolve them.

Authors should not be blocked indefinitely. If a reviewer's concern is genuine but not urgent, it can be filed as follow-up work rather than holding a correct change out of the codebase.

## Keeping Changes Small

Small changes are reviewed faster, reviewed better, and are easier to revert when something goes wrong. A large change tends to receive shallow review because the effort required to review it thoroughly is high.

Where a change must be large, separating mechanical refactoring from behavioural change into distinct commits lets a reviewer verify the mechanical part quickly and concentrate on the part that carries risk.

## **Review Turnaround Expectations**

Responsiveness matters more than depth for the first response. Acknowledging a review promptly, even to say when it will be looked at, unblocks the author's planning far more than a thorough review that arrives two days later.